



Conversational business intelligence, where **you** decide what the model sees.

Ask your database a question in plain language and get a written answer, a table and a chart. Every query is checked before it runs, and every connection controls exactly how much of your data reaches the LLM.

PostgreSQL

MySQL

SQL Server

Oracle

Anthropic

OpenAI

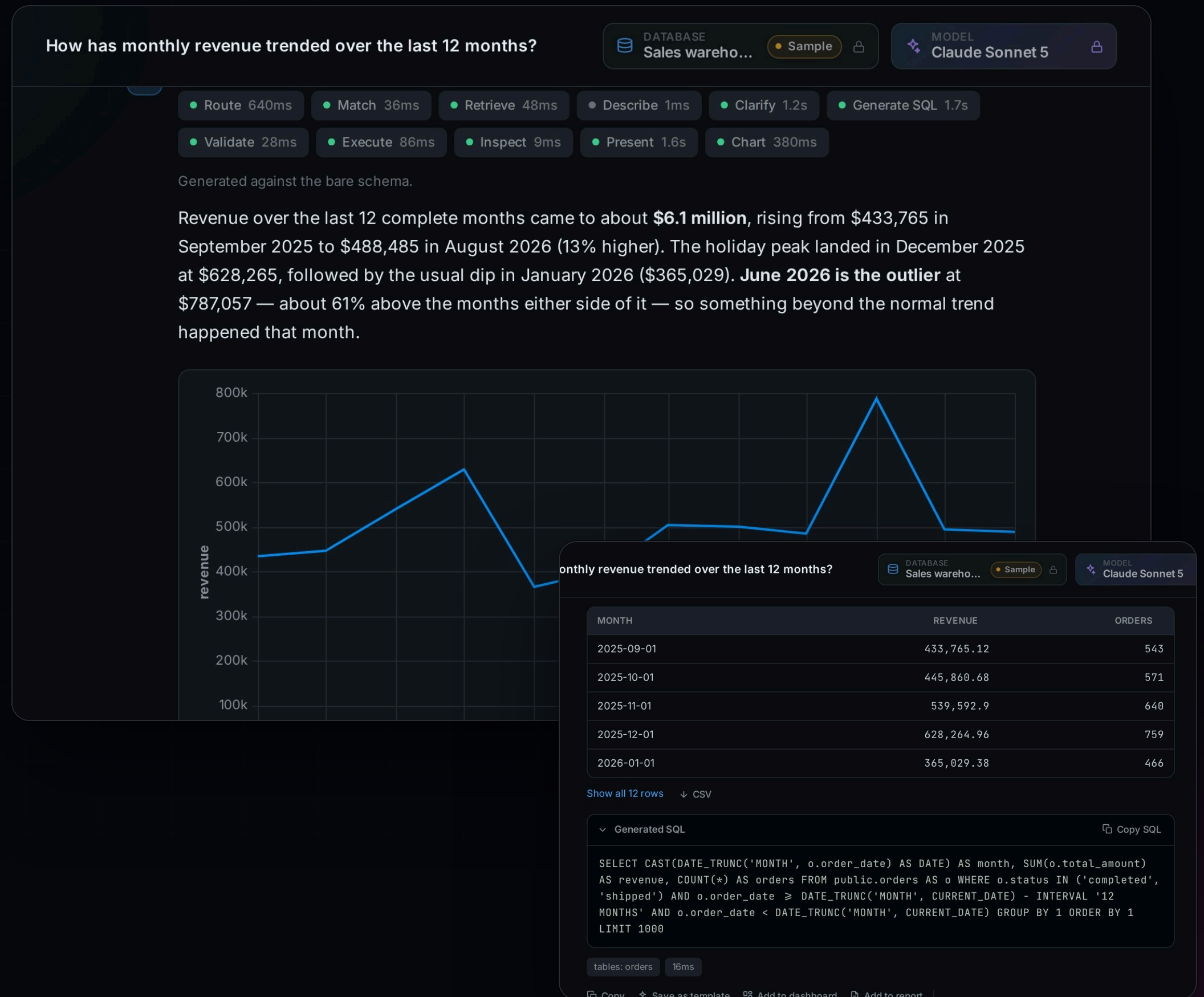
Local models



01 • ASK

Ask the way you'd ask a colleague

- DataMind finds the right tables, **writes the SQL**, **checks it**, and runs it on a read-only connection. Every step is visible as it runs.
- In seconds: a **written answer, a table and a chart**.
- The **exact SQL is always shown**, so anyone can audit the result.
- Follow-ups understand the conversation, and any chart can be redrawn **without re-running the query**.



THE DATAMIND DIFFERENCE

You decide **what the model sees**

When an AI answers questions about your data, the real question is what the model gets to see. In DataMind, every connection declares a result-sharing policy, and the model gets exactly that much.

Result sharing, set on every connection

What a query result may send to the model provider

● NONE

Nothing

The model never sees a single row.

🔒 1,412 rows — count only

● AGGREGATE

Totals only

Column names and counts — never a value.

region	revenue	orders

🔒 names · counts · no values

● SAMPLE

A sample of rows

The first rows of a result, up to 50.

region	revenue	orders
Americas	2,462,367	7,812
Europe	1,689,183	5,321

📄 ≤ 50 rows

● FULL

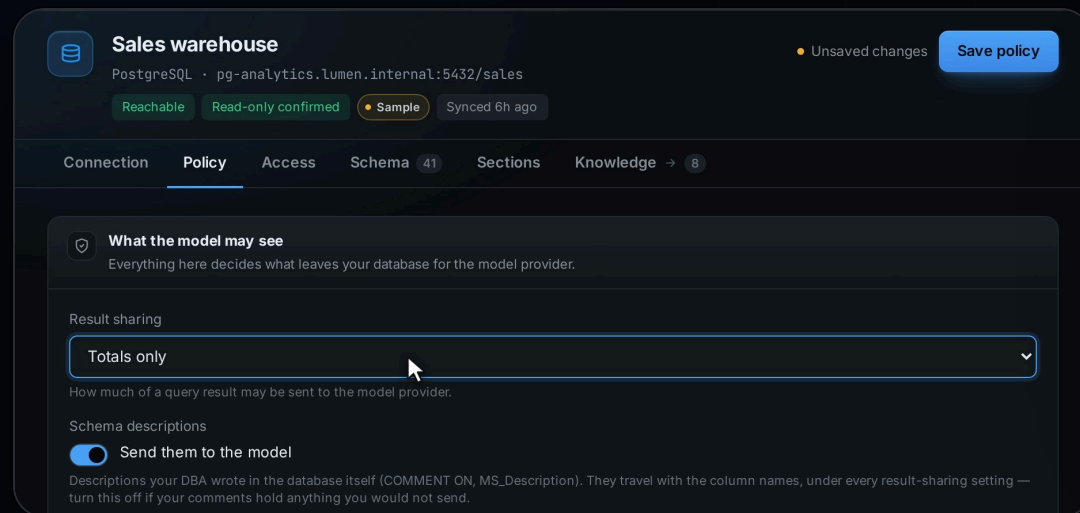
All rows

Every row the query returned.

region	revenue	orders
Americas	2,462,367	7,812
Europe	1,689,183	5,321
APAC	821,988	2,604
Nordics	334,012	1,077
Mid-East	303,550	968

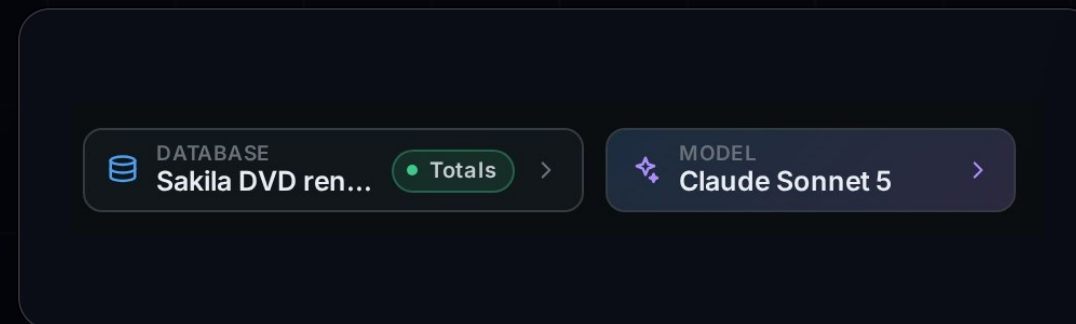
📄 all rows

The policy, in practice



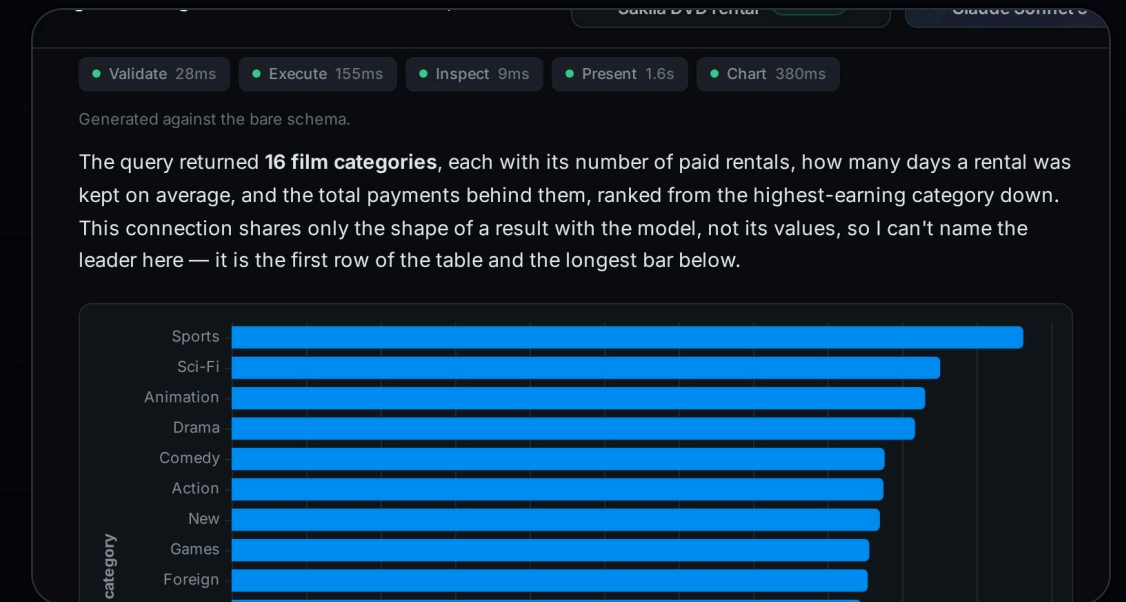
Set on each connection

Nothing, totals only, a sample of rows, or all rows.
Only someone with full access can widen it, and every change lands in the audit log.



Visible when you ask

The policy in force sits in the chat header at the moment you ask, so nobody has to guess what leaves the database.



Answers without the values

Under “totals only”, the model writes its answer without ever seeing a value. The numbers stay in the table, on your screen.

03 • CHECKED

Every query is checked before it runs

- The model only **proposes** SQL. It never executes anything.
- Each statement is parsed into an AST and checked against an **allowlist and the real schema**. Anything unrecognised is a rejection, not a warning.
- A rejected draft goes back for **automatic repair**, as here: a column that doesn't exist, caught and fixed.
- Underneath: **read-only execution**, a row cap and a statement timeout.

Asia Pacific	821,988.23
Nordics	334,011.73
Middle East	303,549.59

[Show all 8 rows](#) ↓ CSV

Generated SQL

1 repair

[Copy SQL](#)**Attempt 1 — rejected**

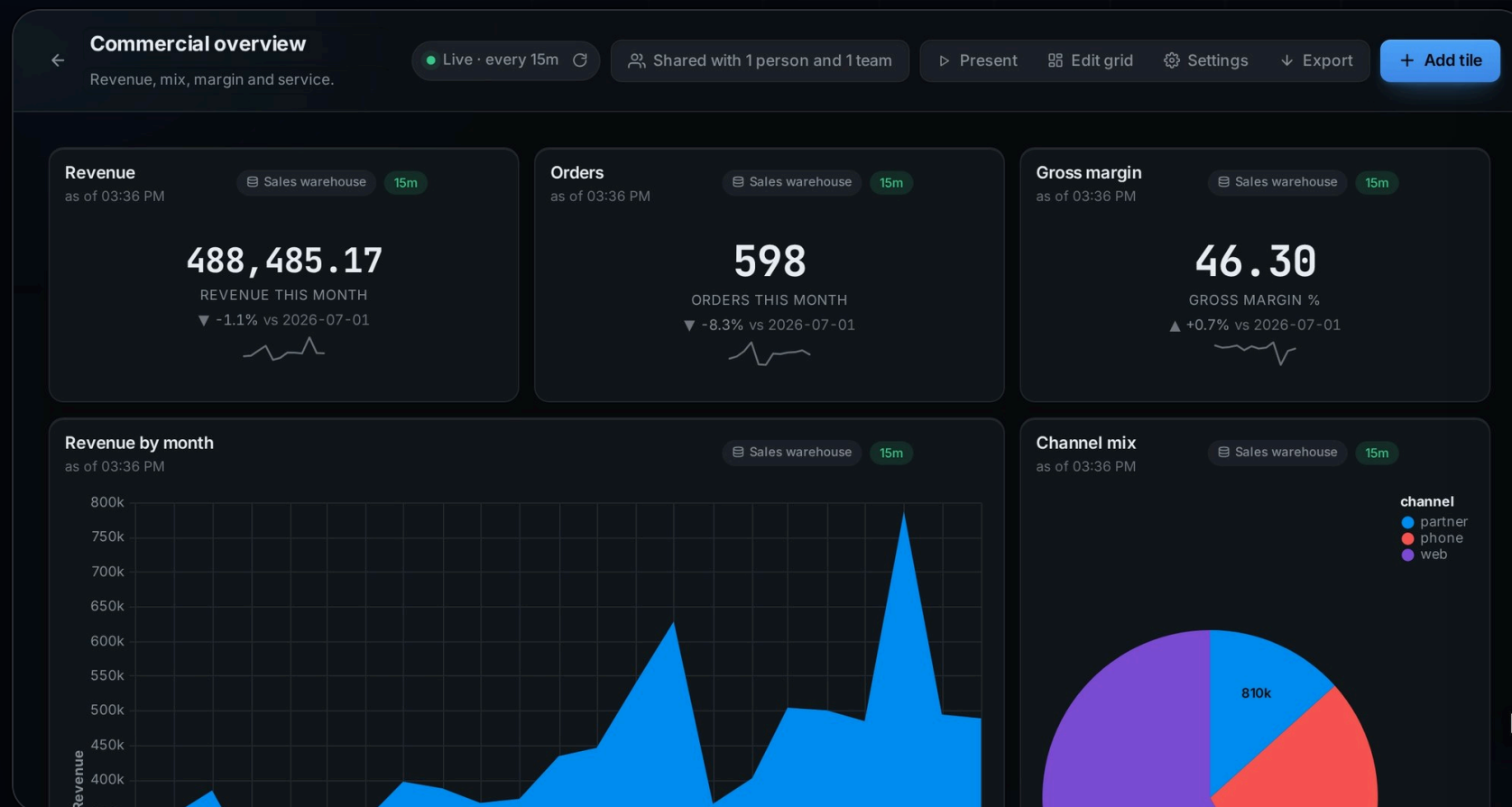
```
SELECT c.region AS region,
       SUM(o.total_amount) AS revenue
FROM public.orders AS o
JOIN public.customers AS c ON c.id = o.customer_id
WHERE o.status IN ('completed', 'shipped')
      AND o.order_date ≥ date_trunc('month', CURRENT_DATE) - INTERVAL '12 months'
      AND o.order_date < date_trunc('month', CURRENT_DATE)
GROUP BY 1
ORDER BY revenue DESC
```

[E_UNKNOWN_COLUMN] Column 'region' does not exist on public.customers.

```
SELECT r.name AS region, SUM(o.total_amount) AS revenue FROM public.orders AS o JOIN
public.customers AS c ON c.id = o.customer_id JOIN public.regions AS r ON r.id =
c.region_id WHERE o.status IN ('completed', 'shipped') AND o.order_date ≥
```

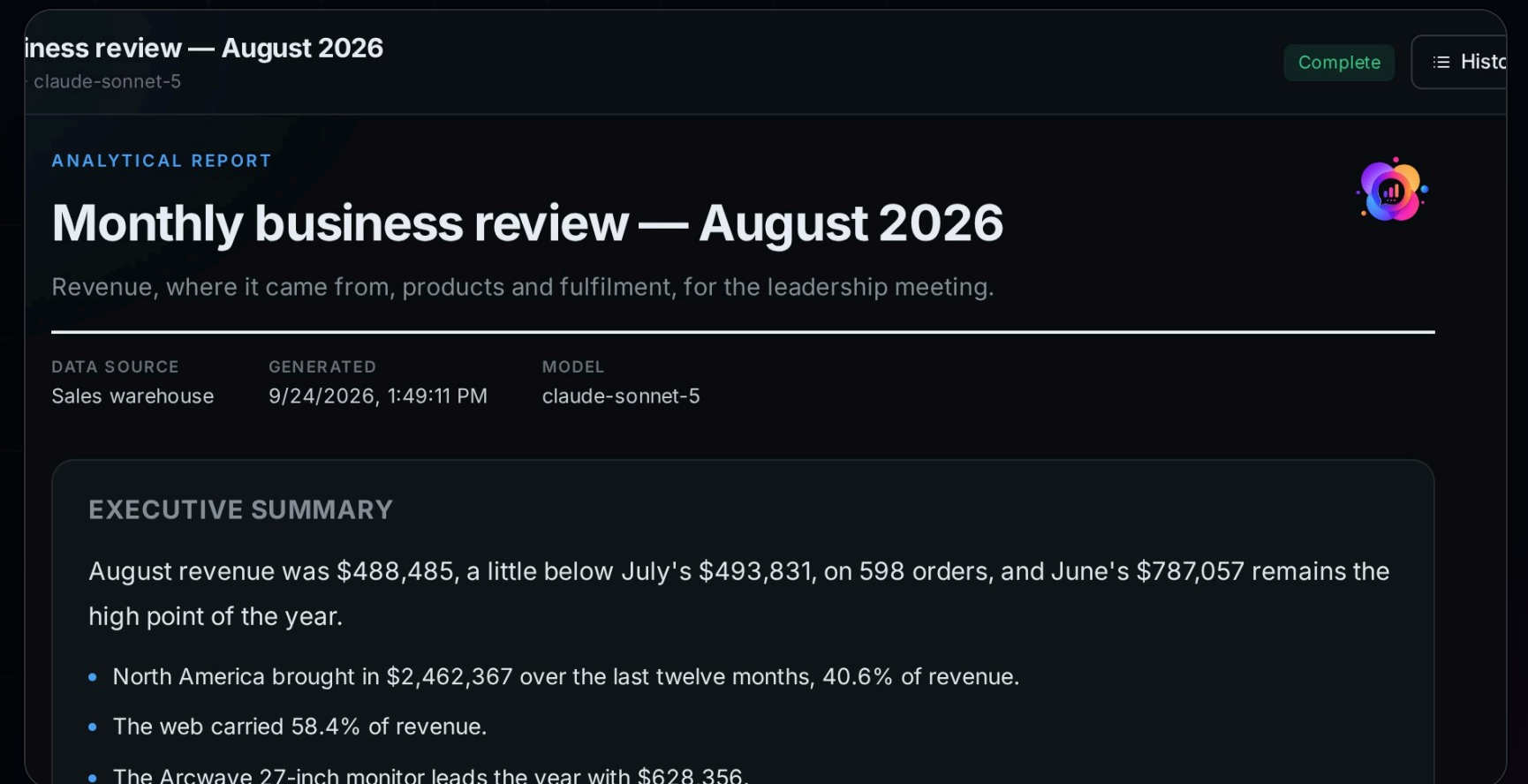
04 • KEEP

Dashboards and reports, on the same guarded path



Dashboards that stay current

Each tile is a saved, checked query with its own connection and refresh rate. **No model call at refresh time**, so boards keep working without an AI provider.



Reports written over your data

Describe a document, approve the outline, get prose written over real results. **Headline figures are computed from the rows**, not by the model. Print it, or re-run it next month.

05 • LEARN

It learns how your business asks

- A question answered correctly can be **taught**: a curator saves it as a verified, parameterised query.
- Similar questions are then **answered from the verified query**, consistently, and still through the guard.
- An optional **semantic layer** (metrics bound to exact SQL, business terms, time conventions) makes answers follow your definitions.

Sales warehouse
pg-analytics.lumen.internal:5432/sales
8 suggested

Store access Connection settings

Taught 6 Suggested 8

Check the store Teach a question

Q Search taught questions

Word matching
Questions are matched on the words they share. Embedding search also matches ones that mean the same thing in different words. No model provider is set up to embed yet — give one an embedding model in LLM providers.

- ✓ Monthly revenue for {region}
public.customers, public.orders, public.regions 41 hits
- ✓ Top 10 customers by revenue this year
public.customers, public.orders
- ✓ Average delivery time for {carrier}
public.carriers, public.shipments

All 7 steps passed · 3.1s

- Route 640ms
- Match 64ms
- Validate 28ms
- Execute 82ms
- Inspect 9ms
- Present 1.6s
- Chart 380ms

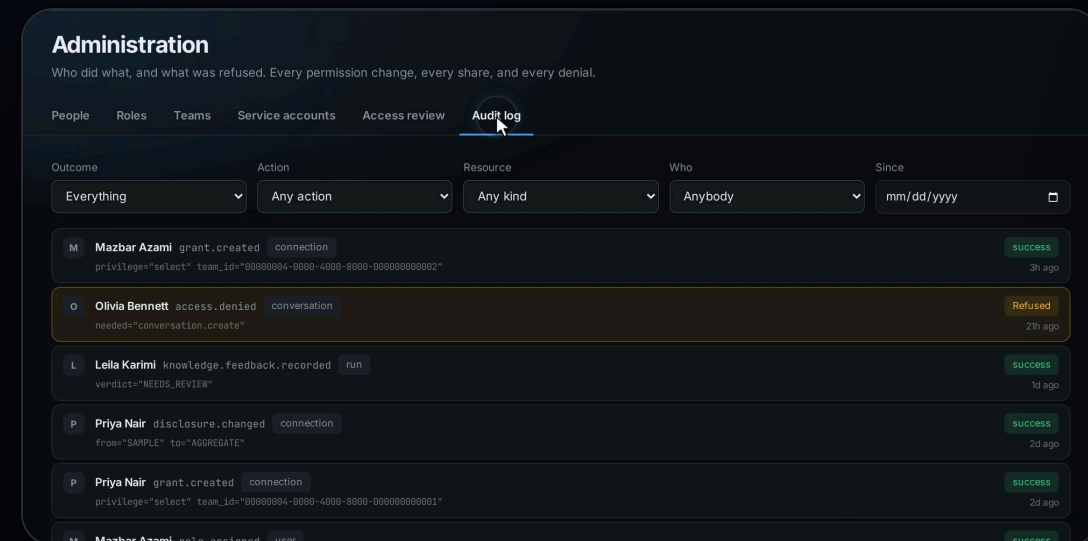
✓ **Verified** answered from a saved question
"Monthly revenue for {region}" · region=Europe
Not what you asked? [Generate a fresh answer instead](#)

Europe brought in \$1,689,183 over the last 12 complete months, from 1,894 orders. Its best month was March 2026 at \$173,334 and its quietest January 2026 at \$100,121; it went from \$142,919 in September 2025 to \$128,264 in August 2026. This is the saved question "Monthly revenue for {region}" with Europe filled in, so it counts booked orders by the customer's region, exactly as the analytics team defined it.

180k

06 • ORGANISATION

Ready for the whole organisation



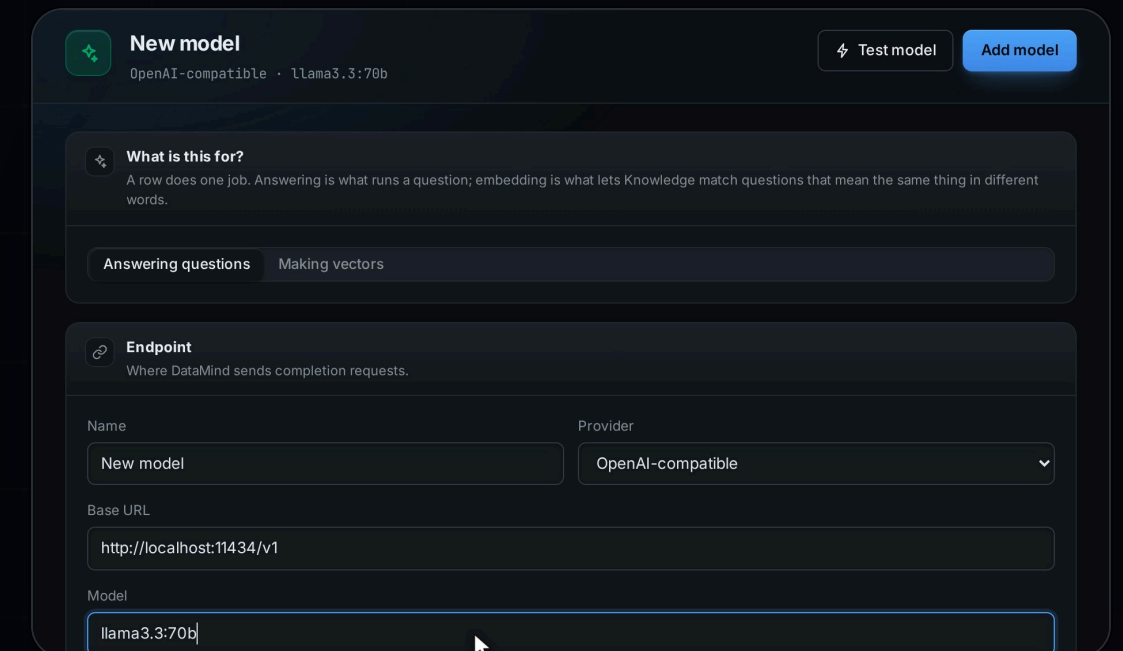
Access control and audit

Roles, teams and fine-grained sharing on every connection, dashboard and report, with an audit log of every permission change, share and denial.



Token usage

What the models used, per person and per model, over any period. Counts only: no question, answer or SQL on the screen.



New model OpenAI-compatible · llama3.3:70b [Test model](#) [Add model](#)

What is this for?
A row does one job. Answering is what runs a question; embedding is what lets Knowledge match questions that mean the same thing in different words.

Answering questions Making vectors

Endpoint
Where DataMind sends completion requests.

Name: New model Provider: OpenAI-compatible

Base URL: http://localhost:11434/v1

Model: llama3.3:70b

Your databases, your models

PostgreSQL, MySQL, SQL Server and Oracle. Anthropic, OpenAI, or a local model on any OpenAI-compatible server, so nothing has to leave your network.

UNDER THE HOOD

Built to be trusted with production data

EVERY CHAT QUESTION RUNS THIS GRAPH

route → match → scope → retrieve → describe → clarify → generate → **validate** → **execute** → inspect → present → chart

Architecture

- Modular monolith: **FastAPI** and SQLAlchemy 2 (async) on PostgreSQL
- **LangGraph** orchestrates runs; **LiteLLM** sits behind a gateway port
- **SQLGlue** guard, dialect-aware; **React** + TypeScript with Vega-Lite charts
- Ports and adapters at four seams: LLM, database, secrets, run execution

Engineering

- ~1,790 backend tests; a **hostile-SQL corpus** is a hard CI gate
- Import-linter contracts enforce the dependency rule
- Argon2id passwords, rotating refresh tokens, AES-256-GCM secrets
- An offline eval harness scores the real pipeline against a fixture database



DataMind

Answers from your data, on your terms.

Ask in plain language

Every query checked

You decide what the model sees

Live demo mazyar375.github.io/DataMind

Source github.com/mazyar375/DataMind

Screens are from the public demo: sample data, the real pipeline.